

Дмитрий Лёмин

Senior Frontend / Full-Stack Engineer · React · TypeScript · Node.js · LLM-системы

leminds@gmail.com · github.com/lemind · linkedin.com/in/dmitrii-lemine-45013858 · t.me/Lemind

Senior-инженер с 10+ годами опыта в frontend-разработке (React, Next.js, TypeScript) в fintech, трейдинге и enterprise — включая руководство командой и запуск продуктов с нуля — и **практическим full-stack опытом** на Node.js/NestJS. Сейчас развиваюсь в сторону **LLM-инженерии**: самостоятельно разработал и вывел в прод Biassemble — систему аудита рассуждений на LLM (TypeScript-ядро + Python RAG-движок) с выстроенным процессом оценки качества: golden-датасеты, CI-гейты по метрикам, версионирование промптов.

ГЛАВНЫЙ ПРОЕКТ — BIASSEMBLE (СИСТЕМА АУДИТА РАССУЖДЕНИЙ НА LLM)

Система для поиска когнитивных искажений в тексте, разработанная в одиночку и развёрнутая в продакшене, с **привязкой к дословным цитатам** — для каждого найденного искажения обязательна ссылка на точный фрагмент исходного текста (схема валидируется через Zod), а результат «искажений не найдено» считается полноценным успешным исходом, а не провалом. Три архитектуры поиска (retrieval) за два месяца — каждый переход обоснован измерениями, а не вкусовщиной.

- **Архитектура**: оркестрация на TypeScript/Fastify · поисковый (retrieval) движок на Python/FastAPI · pgvector/Supabase · мультипровайдерный LLM-слой (Gemini, Anthropic, OpenAI) · база из 38 искажений с версионированием контента (без переписывания истории)
- **Эксперименты с поиском — с цифрами на руках**: baseline на чистом векторном поиске застрял на Recall@5 0.667 после четырёх экспериментов с базой знаний; стадия zero-shot NLI подняла eval Recall@5 до 0.875, но была заменена гибридом (генеративный LLM и векторный поиск), когда задержка NLI в 67 с на CPU оказалась непригодной для продакшена
- **Асинхронная архитектура**: retrieval запускается в момент отправки истории и работает во время 30–120 с, которые пользователь тратит на ответы на уточняющие вопросы — 67 секунд задержки модели стали незаметны без апгрейда железа
- **Дисциплина оценки**: read-only golden-датасеты (positive/edge/adversarial/negative), CI-гейты: false-positive rate < 0.10 и evidence-grounded rate ≥ 0.90, пересчёт порогов после каждой переиндексации, разрыв на проде отслеживается открыто (0.562 против цели 0.85)
- **Дообученная (fine-tuned) модель отбора искажений (LoRA поверх Gemma-3-4B)**: обучена на 900+ синтетических и продакшен-примерах с кросс-модельной верификацией; +71% относительный прирост recall и почти 3-кратный рост precision на статистически значимой отложенной выборке (N=80), без единой регрессии — выкачена в прод после пересмотра критерия готовности с метрики на 4 примерах на метрику на 80, что стало более обоснованным решением
- **Работа с AI-инструментами**: разработка велась с использованием AI-ассистентов (Cline + DeepSeek/Qwen) как источника гипотез — вся архитектура, методология оценки и качественные гейты спроектированы и приняты мной; каждый значимый артефакт проходит ревью минимум двух независимых моделей

Ссылки: [демо](#) · [исходный код движка](#) · статья: «Building a RAG Engine: Three Engineering Assumptions I Had to Unlearn» (dev.to)

НАВЫКИ

AI / LLM-инженерия: RAG, embeddings + pgvector, zero-shot NLI, prompt engineering и версионирование, методология оценки (golden-датасеты, Recall@K, FP-гейты), оркестрация LLM (Gemini / Anthropic / OpenAI), структурированный вывод (Zod), Python, FastAPI, LoRA fine-tuning (QLoRA, PEFT)

Frontend: React, Next.js, TypeScript, JavaScript, GraphQL, Tailwind, CSS Modules, styled-components

State & data: TanStack Query, Redux Toolkit, MobX, Jotai, Recoil

Backend & API: Node.js, NestJS, Express, Fastify, PostgreSQL, MongoDB, REST

Тестирование: Jest, Testing Library, Vitest, Cypress

Архитектура и практики: компонентная архитектура, Feature-Sliced Design, оптимизация производительности, SOLID, основы accessibility (a11y)

Инструменты: Git, Vite, Webpack, Docker, OpenShift, GitOps, CI/CD, Agile/Scrum; React Native, Cordova (мобильная разработка)

ОПЫТ РАБОТЫ

Senior Frontend Engineer · криптотрейдинговая платформа · ~20 frontend-инженеров *март 2025 – настоящее время*

React, Next.js (App Router, SSR), TypeScript, GraphQL, Strapi, Node.js, OpenShift

- Отвечал за аутентификацию и работу с аккаунтами (регистрация, KYC, 2FA, API-ключи) в критичных пользовательских сценариях; устранил edge-case проблемы в NextAuth, из-за которых пользователи теряли сессию во время активной торговли

- Реализовал редизайн личного кабинета, упростив управление аккаунтом и повысив конверсию из регистрации в первую транзакцию
- Разрабатывал контентные страницы на App Router + SSR (Strapi), сократив время начальной загрузки; провёл рефакторинг ключевых частей кодовой базы на Feature-Sliced Design
- Разработал и поддерживал внутреннюю админ-панель (Radix + Refine, 6+ страниц); расширял backend на Node.js; управлял feature flags и конфигурацией окружений через OpenShift/GitOps

Senior Frontend Engineer · трейдинговая платформа · 8–12 frontend-инженеров · дек 2023 – фев 2025

TypeScript, React, React Native, TanStack Query, Jest, styled-components

- Разрабатывал функциональность для действующей трейдинговой платформы, включая сценарии открытия инвестиционного счёта, напрямую влияющие на конверсию в активных инвесторов
- Внедрил кастомный TanStack Query DSL для унификации серверного и клиентского состояния; стандартизировал паттерны работы с API в команде
- Реализовал адаптивный редизайн ключевых пользовательских сценариев; поддерживал покрытие тестами (Jest/Testing Library) на критичных участках; участвовал в разработке на React Native

Senior Frontend Engineer · Swift Inventions (Vierally / ProprOffr / MeeSmartWallet) · 3–4 инженера · дек 2022 – ноя 2023

TypeScript, React, Tailwind, Vite, Jotai/Recoil, NestJS, PostgreSQL, Astro, Stripe

- Разрабатывал frontend с нуля для трёх продуктов в стартап-среде; довёл MVP от концепции до релиза для ранней валидации гипотез
- ProprOffr: основной frontend для MVP платформы аукционов недвижимости с вниманием к доступности (a11y); участвовал в разработке backend-модулей (NestJS, PostgreSQL) и интеграции Stripe
- Реализовал виртуализацию списков и оптимизацию рендеринга для насыщенных данными интерфейсов; настроил тестирование на Vitest в Vierally

Team Lead / Senior Frontend Engineer · Jetlyn

март 2020 – ноя 2022

React, Redux Toolkit, Cordova, Jest

- Разработал с нуля frontend корпоративной системы заявок в IT со сложными многошаговыми процессами согласования; вырастил frontend-команду с 1 до 3 инженеров, отвечал за декомпозицию задач и код-ревью
- Разрабатывал white-label банковские приложения для нескольких клиентов; интегрировал СБП (систему быстрых платежей); выпустил мобильные функции на Cordova

Более ранний опыт — становление во frontend

2006 – 2020

- Прошёл путь от веб-разработки до современной frontend-инженерии: SPA, маркетплейсы, админ-системы; внедрил e2e-тестирование на Cypress; перевёл легаси-стек на современную архитектуру; код-ревью и менторинг

ОБРАЗОВАНИЕ И КУРСЫ

Бакалавр, Информатика и вычислительная техника — Пензенский государственный университет, 2001–2006

Курсы по Generative AI / LLM-инженерии (Coursera, IBM и др.) — в т.ч. Generative AI Applications with Python, LLM Architecture & Data Preparation, Language Modeling with Transformers, Prompt Engineering, Python for Data Science & AI

Языки: английский — профессиональный уровень · русский — родной